Editor defects: handoff for a fix session in the app repo

Found by the Playwright suite TC0028-TC0088, Chromium, deployed Workspace app, QA account. Generated 2026-10-12.

HOW TO USE THIS DOCUMENT

- Open a Claude Code session in the **app repo** (AzumoHQ/notion-knockoff-mvp, rich editor under src/features/editor/rich/ (code read at commit b599731 of branch personal/main)). The automation repo is `ai-notion-knockoff-automation`; it contains the failing tests and is not changed by the fix.
- Four application defects, all severity medium, each with steps, expected, actual, screenshot, root cause read from the code, proposed fix and acceptance criteria.
- The tests are failing on purpose and annotated `defect`. Do not weaken them: fix the app, then run them again from the automation repo against the fixed deployment.
- The root causes were found by reading the code at the commit named above. Check that the app branch you work on still has the same lines before editing (file:line references may shift).

ID	Test	Summary	Code area (app repo)
DEF-01	TC0033	Plain-text Markdown mermaid fence stays code	rich/controller.ts:191 vs :127
DEF-02	TC0039	Inline fragment over a word splits the paragraph	rich/controller.ts:146 insertAtCursor
DEF-03	TC0047	Chained table operations use a stale table	rich/TableControls.tsx
DEF-04	TC0067	Undo after a late upload: Reintentar does nothing	rich/controller.ts:91-111

ENVIRONMENT AND CONSTRAINTS

- Spanish UI strings: "Guardado", "Deshacer", "Reintentar", "Agregar fila", "Quitar última columna", "Seleccionar bloques".
- Editor: BlockNote/ProseMirror, `[data-editor="rich"] .ProseMirror`; custom blocks wsImage, wsFile, wsDiagram.
- Pastes in the tests are synthetic (DataTransfer + ClipboardEvent), which exercises the app's paste handler; native-clipboard tests TC0085-TC0086 pass separately.
- Tests create pages named `qa-auto-*` on the QA account and purge them. Do not run destructive flows against production data.
- Flakiness note: the page-creation setup sometimes fails with "Título de la página ... element is not editable" (the title textarea stays readonly for a while after creating a page). It is environmental and independent of these defects; repeat the run.

DEF-01: Markdown pasted as plain text: a ```mermaid fence stays a code block instead of becoming a diagram

Medium TC0033 Paste, plain-text Markdown path

STEPS TO REPRODUCE (MANUAL)

1. Log in as the QA user and open any empty page.
2. Focus the editor body.
3. Paste ONLY plain text (no text/html on the clipboard) with the content below.
4. Look at the block that holds the mermaid fence.

INPUT DATA

````
Before the code QA

```javascript
const total = [1, 2, 3].reduce((a, b) => a + b, 0);
console.log(total);
```

```mermaid
flowchart LR
  A[Start] --> B[Process]
  B --> C[End]
```

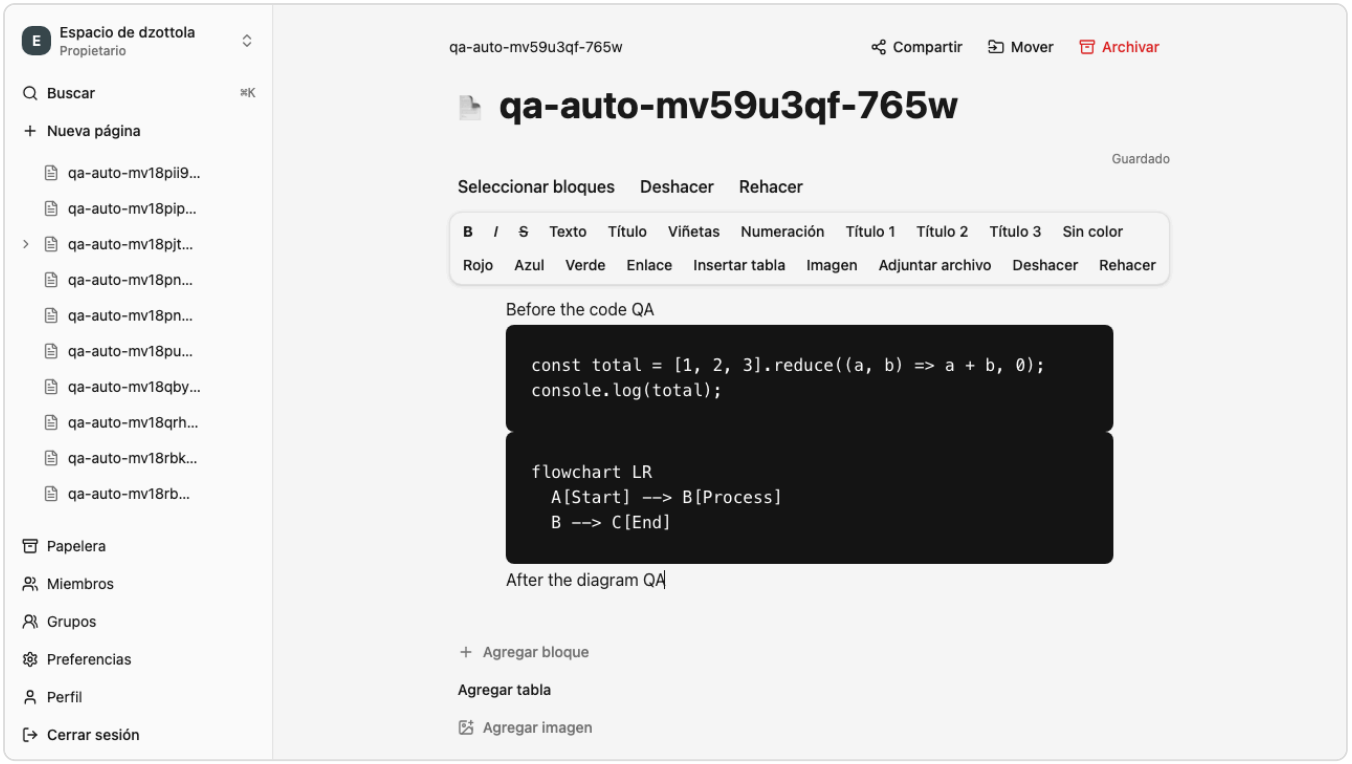
After the diagram QA
````

Expected. The mermaid fence becomes a diagram block (wsDiagram, an SVG with role="img"), exactly as when the same content is pasted as HTML. The javascript fence stays a code block.

Actual. Both fences are plain code blocks. No wsDiagram exists; the mermaid source is shown as text. Pasting the HTML equivalent does render the diagram, so the two paste paths disagree.

EVIDENCE

Screenshot below: second dark block shows "flowchart LR A[Start] --> B[Process]..." as code. The locator [data-content-type="wsDiagram"] [role="img"] svg is never found in 20 s.



ROOT CAUSE

CONFIRMED by reading code. controller.ts:191: when the clipboard has no HTML and no files, createPasteHandler returns defaultPasteHandler({ plainTextAsMarkdown: looksLikeMarkdown(text) }), i.e. BlockNote's own Markdown import. The codeBlock(language "mermaid") -> wsDiagram conversion exists only in planToBlocks (controller.ts:127), which runs on the HTML path. The plain-text Markdown path never reaches it.

PROPOSED FIX

Route Markdown text through the same conversion: when looksLikeMarkdown(text) is true, convert it with editor.tryParseMarkdownToBlocks(text), run the result through the same mapping used in planToBlocks (extract it to a shared function that maps mermaid codeBlock -> wsDiagram), and insert with insertAtCursor. Keep the plain-text fallback for text that is not Markdown (TC0034 must keep passing).

ACCEPTANCE CRITERIA

TC0033 passes. TC0030-TC0032, TC0034 and TC0049 still pass. Undo after the paste removes the whole pasted content in one step.

AUTOMATED REPRODUCTION

Test: `tests/regression/editor-paste.spec.js:65, fixture data/clipboard/code-mermaid.md` in the automation repo. Run from its root:

```
npx playwright test tests/regression/editor-paste.spec.js -g "TC0033" --project=chromium --retries=0 --workers=1
```

Artifacts of the run used for this report (screenshot, trace.zip, video.webm, error-context.md, log): `docs/defects/evidence/DEF-01/`.

DEF-02: Pasting a single inline fragment over a selected word splits the paragraph into three blocks

Medium TC0039 Paste, insertion at the cursor

STEPS TO REPRODUCE (MANUAL)

1. Open an empty page. Type "One REPLACE three" in one paragraph, then a second paragraph "Final QA". Wait for "Guardado" and reload.
2. Select exactly the word "REPLACE" (characters 4 to 11 of the first paragraph).
3. Paste a clipboard with text/html = NEW and text/plain = NEW.

INPUT DATA

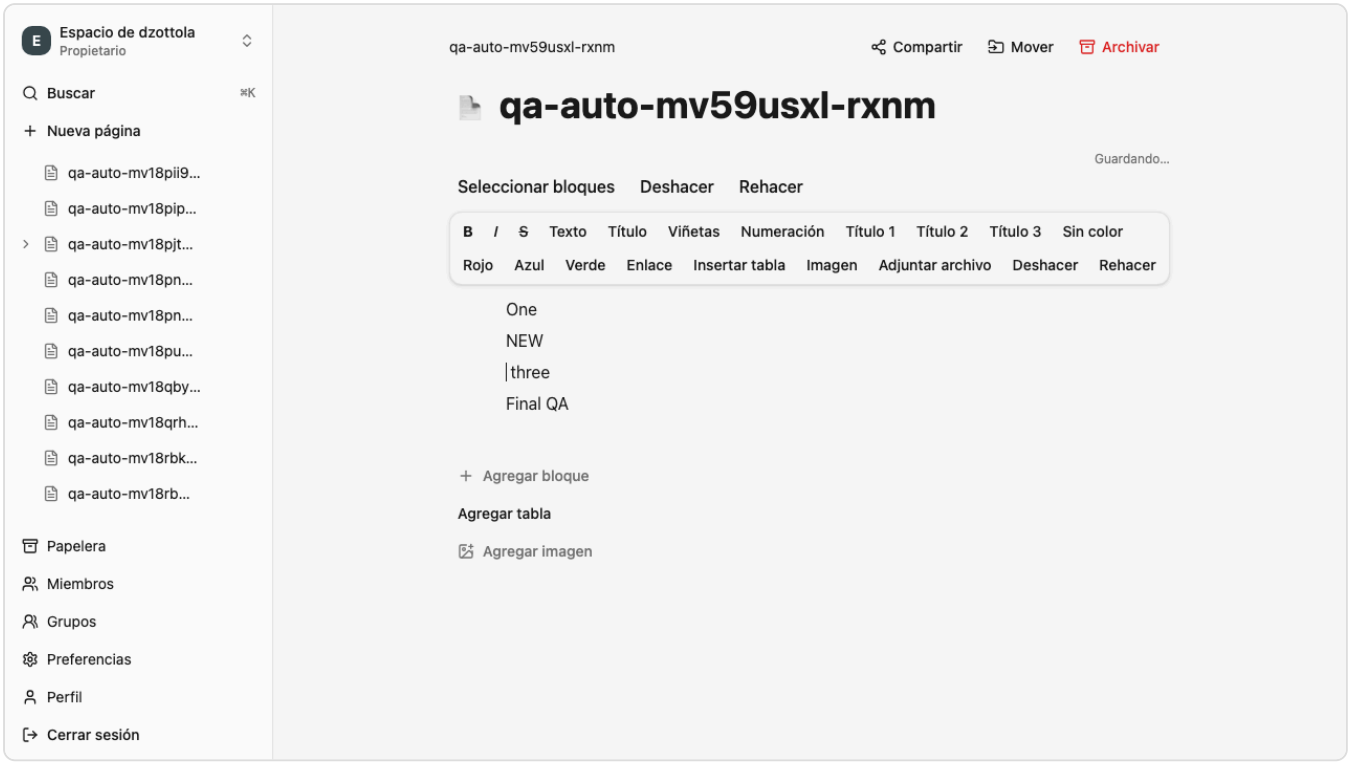
```
text/html:  <span>NEW</span>
text/plain: NEW
```

Expected. The selection is replaced inside the paragraph: ["One NEW three", "Final QA"]. Compare with Notion, Google Docs: an inline fragment never creates new blocks.

Actual. Three separate paragraphs: ["One", "NEW", "three", "Final QA"].

EVIDENCE

Screenshot below. Assertion diff: Expected ["One NEW three","Final QA"], Received ["One","NEW","three","Final QA"].



ROOT CAUSE

CONFIRMED by reading code. controller.ts:146 insertAtCursor: after deleting the selection, if the caret is neither at the start nor at the end of the block, it always runs tr.split(...) and inserts the parsed blocks between the two halves. It does not distinguish a block paste (.. , which should split; TC0037 depends on that) from an inline fragment (a , plain text on one line), which should be inserted into the existing inline content.

PROPOSED FIX

In pasteBlocks/insertAtCursor, detect an inline-only paste (the parsed HTML yields exactly one paragraph and the source html has no block-level element) and insert its inline content at the cursor with editor.insertInlineContent(...), without splitting. Keep the split behaviour for real block pastes.

ACCEPTANCE CRITERIA

TC0039 passes. TC0035-TC0038 and TC0040 (start, end, offset, multi-block) still pass. Reload keeps "One NEW three".

AUTOMATED REPRODUCTION

Test: `tests/regression/editor-paste.spec.js:114` in the automation repo. Run from its root:

```
npx playwright test tests/regression/editor-paste.spec.js -g "TC0039" --project=chromium --retries=0 --workers=1
```

Artifacts of the run used for this report (screenshot, trace.zip, video.webm, error-context.md, log): `docs/defects/evidence/DEF-02/`.

DEF-03: Consecutive table operations act on a stale table (a previous row/column edit is lost)

Medium TC0047 Table toolbar

STEPS TO REPRODUCE (MANUAL)

1. Open an empty page and paste the HTML table below (4 rows x 3 columns).
2. Click inside the cell "One".
3. Click toolbar "Agregar fila" twice. Expected 6 rows.
4. Click toolbar "Quitar última columna".
5. Read the table size.

INPUT DATA

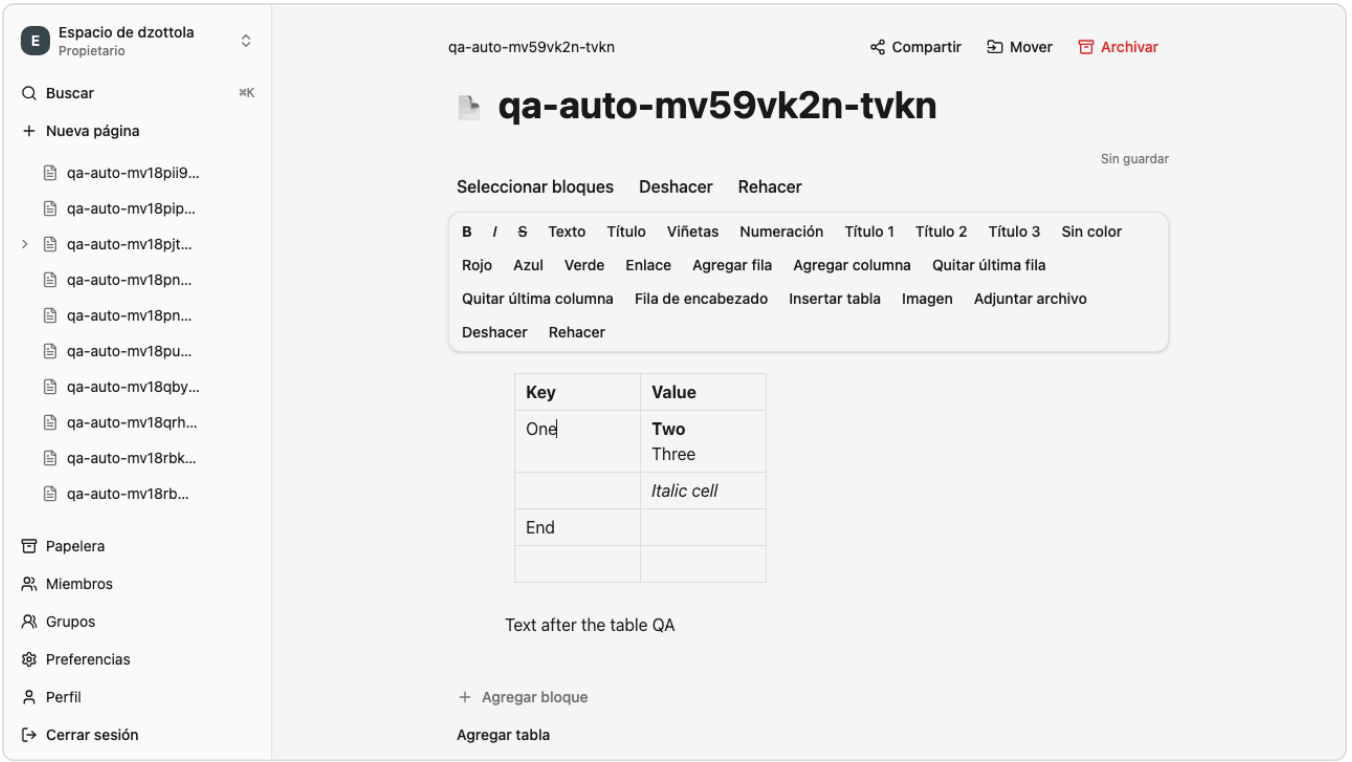
```
data/clipboard/tables.html (4x3 table: header Key|Value|Note, cell with a line break, empty cells, italic and linked cells)
```

Expected. After two "Agregar fila" the table has 6 rows; after "Quitar última columna" it has 6 rows x 2 columns.

Actual. The table ends with 5 rows x 2 columns: one of the previously applied edits is lost. Each button works when used once on a fresh table (TC0042-TC0045 pass), the failure appears only when operations are chained.

EVIDENCE

Screenshot below: 5 rows x 2 columns. Assertion diff at editor-paste.spec.js:194: Expected rows 6, Received rows 5.



ROOT CAUSE

CONFIRMED by reading code. TableControls.tsx reads the table once per render (editor.getTextCursorPosition().block) and every onClick builds the new rows from that captured `table`/`block`. Clicking a button calls editor.updateBlock, which does not move the selection, so the toolbar does not re-render and the next click still holds the old table. Each click therefore overwrites the previous change with a modification of the old snapshot.

PROPOSED FIX

Read the current block inside every handler: const current = editor.getTextCursorPosition().block (and its content) at click time, then compute the new rows from it. Alternatively make the toolbar re-render on editor.onChange. The first option is simpler and has no

render dependency.

ACCEPTANCE CRITERIA

TC0047 passes and TC0042-TC0046 still pass. Also verify by hand: add row, add column, remove row, remove column, toggle header, in any order, each applied on top of the previous result.

AUTOMATED REPRODUCTION

Test: `tests/regression/editor-paste.spec.js:185` in the automation repo. Run from its root:

```
npx playwright test tests/regression/editor-paste.spec.js -g "TC0047" --project=chromium --retries=0 --workers=1
```

Artifacts of the run used for this report (screenshot, trace.zip, video.webm, error-context.md, log): `docs/defects/evidence/DEF-03/`.

DEF-04: Undo after an upload finished while its image was deleted leaves an error placeholder whose "Reintentar" does nothing

Medium TC0067 Image paste, upload lifecycle and undo

STEPS TO REPRODUCE (MANUAL)

1. Open an empty page. Hold the Storage upload in flight (in the automated test: page.route on **/storage/v1/object/** POST, awaiting a gate).
2. Paste a PNG image (clipboard file). A placeholder appears with the uploading state.
3. While it is uploading, delete the placeholder (Seleccionar bloques, check the block, Eliminar).
4. Release the upload and wait for the POST to finish and the status to read "Guardado".
5. Click "Deshacer" (undo).
6. If the block shows "No se pudo subir la imagen. Reintentar Quitar", click "Reintentar".

INPUT DATA

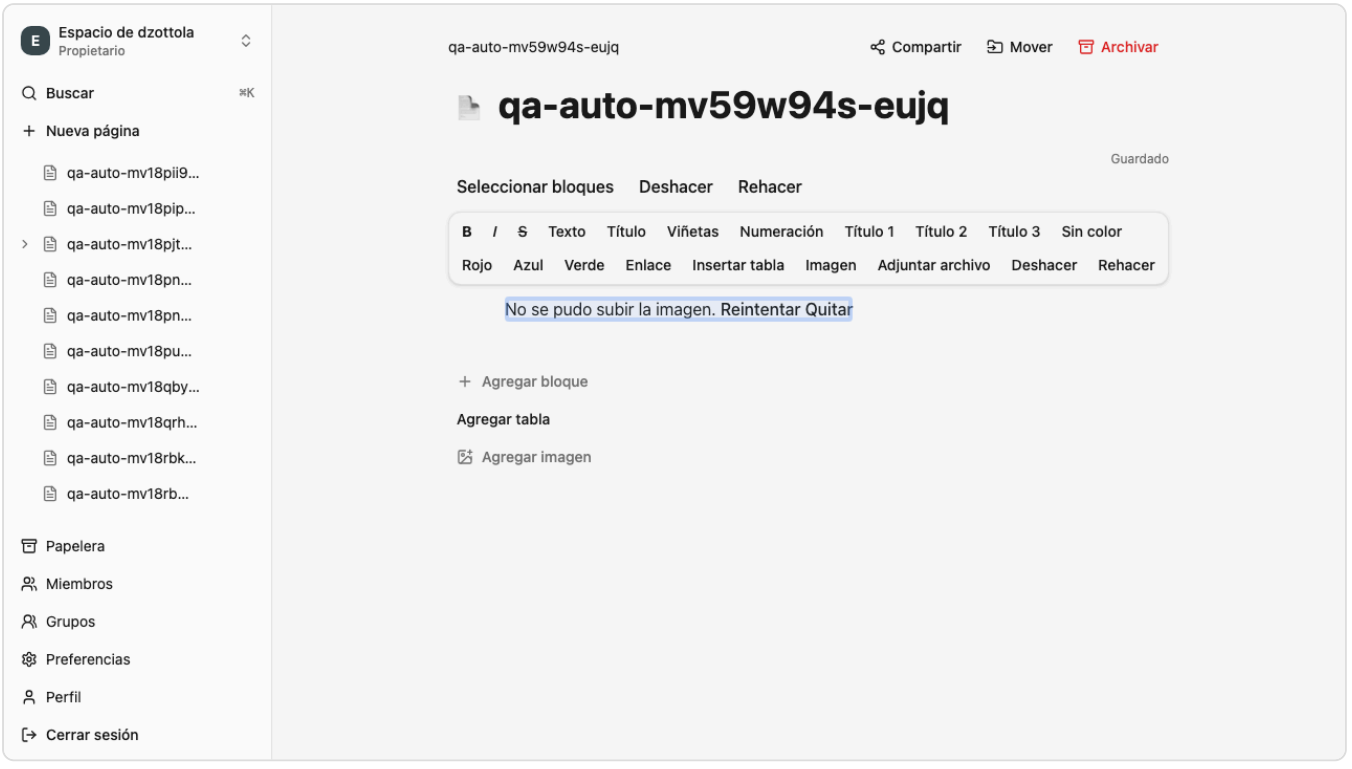
```
A 1x1-colour PNG named late.png pasted as a clipboard file (pngPayload("green","late.png")).
```

Expected. Undo brings back a usable image, or at least an error the person can recover from: "Reintentar" must upload the file again and end with a visible image that survives a reload.

Actual. Undo shows "No se pudo subir la imagen. Reintentar Quitar". Clicking "Reintentar" does nothing: no upload starts and the image never appears (waited 20 s).

EVIDENCE

Screenshot below: the error placeholder after clicking Reintentar. No <img> in the editor. Failing assertion editor-files.spec.js:159, expect(editor.surface.getByRole("img")).toBeVisible().



ROOT CAUSE

CONFIRMED by reading code. controller.ts:91-98 startUpload: on success it calls tracker.done(id, false), which DELETES the stored File, and because the block no longer exists it also calls deps.discard(path) (the uploaded object is removed). Undo then recreates the placeholder; reconcilePlaceholders (controller.ts:85) marks it "failed" because the tracker has no job. retryUpload (controller.ts:109-111) returns false as soon as tracker.file(id) is undefined, so Reintentar is a no-op.

PROPOSED FIX

When the upload succeeds but the block is gone, keep the File in the tracker: tracker.done(id, /* keepFile */ !editor.getBlock(id)) and still discard the uploaded object. Then undo restores the placeholder, reconcilePlaceholders flags it failed, and Reintentar re-uploads the kept file. Make sure the kept file is released when the page is left or the placeholder is purged, to avoid a memory leak.

ACCEPTANCE CRITERIA

TC0067 passes. TC0062-TC0066 (failure, retry, undo variants) still pass. After the retry and a reload, the image is loaded (naturalWidth > 0) and no orphan Storage object remains from the first upload.

AUTOMATED REPRODUCTION

Test: `tests/regression/editor-files.spec.js:137` in the automation repo. Run from its root:

```
npx playwright test tests/regression/editor-files.spec.js -g "TC0067" --project=chromium --retries=0 --workers=1
```

Artifacts of the run used for this report (screenshot, trace.zip, video.webm, error-context.md, log): `docs/defects/evidence/DEF-04/`.

Other findings (no failing test to fix)

- **DEF-05 (low).** The attachment "Descargar" link points at the inline URL until the signed URL with `download=` arrives; a cross-origin <a download> cannot force a download before that. TC0071-TC0072 wait for the signed URL. Optional improvement: render the link only when the signed URL is ready.
- **DEF-06 (WebKit only, not reproduced here).** The "navigate away and back" persistence case failed on WebKit in the app repo's own harness; it passes on Chromium (TC0080). Needs a WebKit run to confirm.
- **TC0012 (existing test, automation side, not an app bug).** It still waits for the old textbox "Texto"; the page now uses the rich editor. Selector update needed in the automation repo.